

7.5 实验五：航点修改与路径再规划

7.5.1 实验目的

本实验旨在掌握飞行任务执行过程中对关键航点进行更新的方法，使飞行路径能够随任务需求变化而调整；并掌握通过 MAVROS 重新推送航点至飞控系统并与设置当前航点的流程，实现任务的在线更新与连续执行。理解无人机任务中路径再规划的重要性，并通过代码实现航点修改与任务路径更新。

7.5.2 实验说明

本实验主要涉及飞控任务中的路径规划与航点修改。学生将通过修改既定航点序列中的关键航点坐标，并对相关航点进行必要的几何约束修正，生成更新后的航点序列，确保无人机能够按照新的路径执行任务。实验将基于 `waypoint_cal.cpp` 文件，学生将学会如何在给定的航点序列中修改指定的航点（例如投弹点），并重新计算下一航点的坐标。完成该实验后，学生将能够使用 MAVROS 推送修改后的航点数据，确保飞控系统能够执行新的飞行任务。能够根据目标位置修改航点，并通过计算得到新的航点位置。确保推送的航点数据正确，并能设置当前航点以开始执行新任务。学生能够体会到在实际飞行任务中，如何应对目标位置变化并调整飞行路径。

7.5.3 实验步骤

步骤一：拉取飞控侧任务航点

使用 `mavros_msgs::WaypointPull` 服务，`waypoint_pull_client` 服务客户端对象从飞控系统拉取当前航点列表，确认航点数据已经成功获取。此步骤写在 `waypointSet` 函数里面，此函数 1s 执行一次检查。通过判断是否到达 1 号航点，到达 1 号航点则拉取一次航线，并设置标志位表示已拉取航线。再通过订阅 `/mavros/mission/waypoints` 话题获取航点列表赋值给变量 `waypoints_read_vector`，并把航线上预设的 10 号点经纬度坐标给 `Lon_Scope` 和 `Lat_Scope` 以及预设投弹点 `drop_coordinates_lat`，到达航点与航点列表函数：

```
// 回调函数，当无人机到达一个航点时调用
void Waypoint::waypointReachedCallback(const
mavros_msgs::WaypointReached::ConstPtr& msg)
{
    waypoint_seq.timestamp = formatRosTime(msg->header.stamp);    //读取
到的时间戳
    waypoint_seq.wp_seq = msg->wp_seq;
    waypoint_seq_buf.push_back(waypoint_seq);
    ROS_INFO("Waypoint reached: seq %d", waypoint_seq.wp_seq);
}
```

```

//航点列表回调函数
void waypoints_callback(const mavros_msgs::WaypointList::ConstPtr& msg)
{
    ROS_INFO("Received %lu waypoints.", msg->waypoints.size());

    waypoints_read_vector = msg->waypoints;

    for (const auto& waypoint : waypoints_read_vector)
    {
        //航点列表信息
        ROS_INFO("  Frame: %d", waypoint.frame);
        ROS_INFO("  Command: %d", waypoint.command);
        ROS_INFO("  Current: %d", waypoint.is_current);
        ROS_INFO("  Autocontinue: %d", waypoint.autocontinue);
        ROS_INFO("  Param1: %f", waypoint.param1);
        ROS_INFO("  Param2: %f", waypoint.param2);
        ROS_INFO("  Param3: %f", waypoint.param3);
        ROS_INFO("  Param4: %f", waypoint.param4);
        ROS_INFO("  X: %f", waypoint.x_lat);
        ROS_INFO("  Y: %f", waypoint.y_long);
        ROS_INFO("  Z: %f", waypoint.z_alt);
    }
    //目标航点经纬度
    Lat_Scope = int32_t(waypoints_read_vector[10].x_lat * 10000000);
    Lon_Scope = int32_t(waypoints_read_vector[10].y_long * 10000000);
    //投弹坐标经纬度
    drop_coordinates_lat_int = int32_t(waypoints_read_vector[10].x_lat *
10000000);
    drop_coordinates_lon_int = int32_t(waypoints_read_vector[10].y_long *
10000000);

    ROS_INFO("Lat_Scope: %d, Lon_Scope: %d", Lat_Scope, Lon_Scope);
}

```

步骤二：修改航点经纬度坐标数据

获取目标位置坐标后，修改投弹点经纬度坐标，将 waypoints_read_vector 赋值给 waypoints_send_vector，将计算出来的投弹点航线坐标 drop_coordinates_lat 赋值给 waypoints_send_vector 相应点（即 10 号点）坐标，并根据给出的点的结构体 Point 和两点之间距离函数编写计算延长线坐标函数，根据投弹点和投弹前一个点的坐标，从而计算延长线上投弹点下一个点的坐标(即已知 9 号和 10 号点坐标，求 11 号点坐标)。更新 waypoints_send_vector 相应航点（即 10 号点和 11 号点）的坐标。修改航点函数代码框架：

```

// 定义一个结构体来表示点

```

```

struct Point {
    double x, y;
};

// 函数来计算两点之间的距离
double distance(const Point& p1, const Point& p2) {
    return sqrt((p2.x - p1.x) * (p2.x - p1.x) + (p2.y - p1.y) * (p2.y - p1.y));
}

void changeWaypoint(void){
    int drop_point = 10;

    /*****
    //修改航点代码由学生编写

    *****/
}

```

步骤三：推送修改后的航点列表数据并设置当前航点

使用 `mavros_msgs::WaypointPush` 服务，`wp_push_client` 服务客户端对象将更新的航点数据发送给飞控系统，确保飞控系统能够接收到并更新数据。使用 `mavros_msgs::WaypointSetCurrent` 服务，`wp_set_current_client` 服务客户端对象设置当前航点，确保飞控能根据新的航点开始执行航线任务（即设置 8 号点为当前航点，飞机将立刻飞往 8 号点，并执行 8 号点后续航线任务）。此步骤通知设置标志位使其只执行一次。

步骤四：验证航点修改

在 Mission Planner 中检查航点列表，确认修改后的航点已成功推送。确保修改后的航点数据能够被飞控系统正确接收，参考实验二，可通过后续实际飞行验证路径修改的效果。

把在虚拟机里写好的程序 `coordinate_cal` 包放进 RK3566 的 `catkin_ws` 文件夹下面的 `src` 里面，编译，运行。

参考代码：

```

// 函数来计算延长线上特定距离的点坐标,已知 p1,p2 坐标,求延长线上点 p3
// 的坐标
Point calculate_extended_point(const Point& p1, const Point& p2, double
distance) {
    double dx = p2.x - p1.x;
    double dy = p2.y - p1.y;
}

```

```

double dist = ::distance(p1, p2);

// 计算延长线上点的比例
double ratio = (dist + distance) / dist;

// 计算延长线上点的坐标
Point p3;
p3.x = p1.x + dx * ratio;
p3.y = p1.y + dy * ratio;

return p3;
}
//航点修改函数
void changeWaypoint(void){
    int drop_point = 10;
    waypoints_send_vector[drop_point].frame =
mavros_msgs::Waypoint::FRAME_GLOBAL_REL_ALT; //相对起飞点的坐标
    标系
    waypoints_send_vector[drop_point].command = 16; // 航点
    waypoints_send_vector[drop_point].is_current = false;
    waypoints_send_vector[drop_point].autocontinue = true;
    waypoints_send_vector[drop_point].param1 = 0;
    waypoints_send_vector[drop_point].param2 = 0;
    waypoints_send_vector[drop_point].param3 = 0;
    waypoints_send_vector[drop_point].param4 = 0;
    waypoints_send_vector[drop_point].x_lat = drop_coordinates_lat;
    waypoints_send_vector[drop_point].y_long = drop_coordinates_lon;
    waypoints_send_vector[drop_point].z_alt = 20;
    Point point1 =
{waypoints_send_vector[drop_point-1].x_lat, waypoints_send_vector[drop_point
-1].y_long };
    Point point2 =
{waypoints_send_vector[drop_point].x_lat, waypoints_send_vector[drop_point].
y_long };
    Point point3 = calculate_extended_point(point1, point2, 0.0004);
    //推送修改后的 11 号航点
    waypoints_send_vector[drop_point+1].x_lat = point3.x;
    waypoints_send_vector[drop_point+1].y_long = point3.y;
}

```